

# DSA1101

Introduction to Data Science

Week 6 Tutorial 4

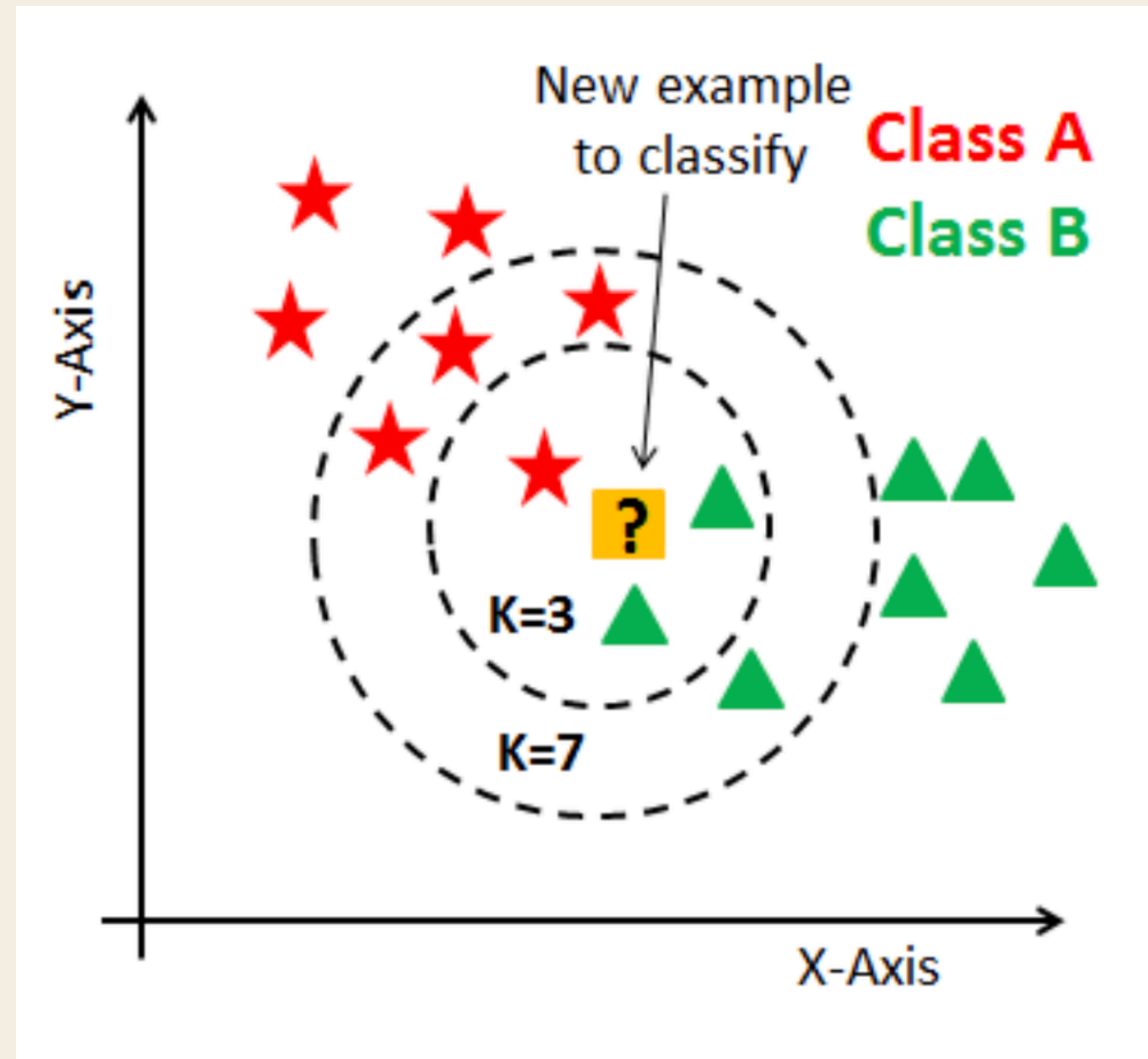
# TODAY'S AGENDA

1. Quick Revision
2. On-site Questions Attempt
3. On-site Questions Discussion
4. Off-site Questions Discussion

# QUICK REVISION

# K-Nearest Neighbours (KNN)

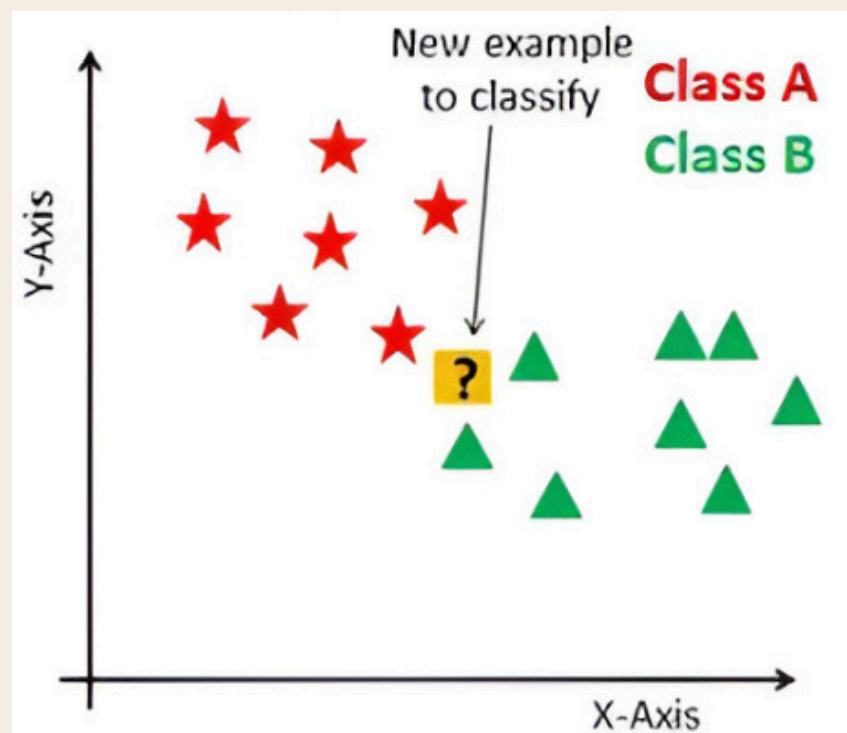
How would you classify the new example?



# K-Nearest Neighbours (KNN)

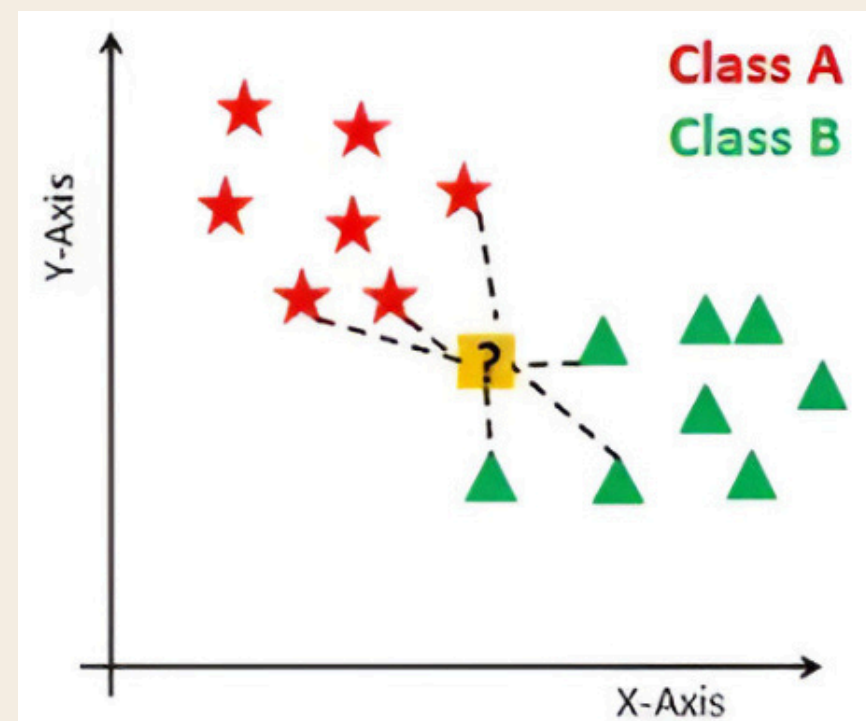
## Step 1

Initialize new test data point



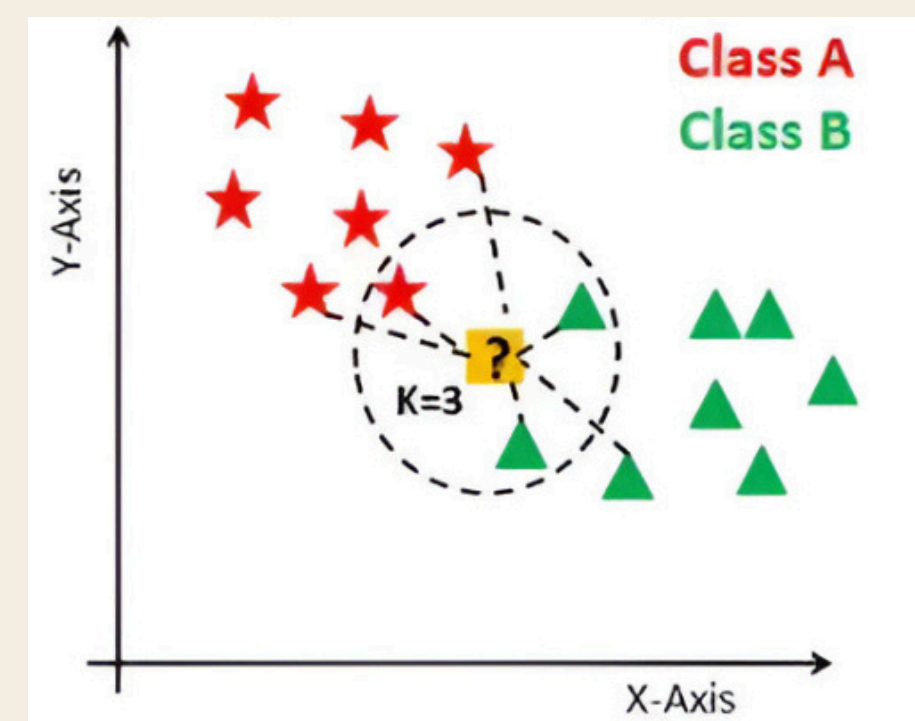
## Step 2

Calculate the euclidean distance between new test data point with **all** train data points



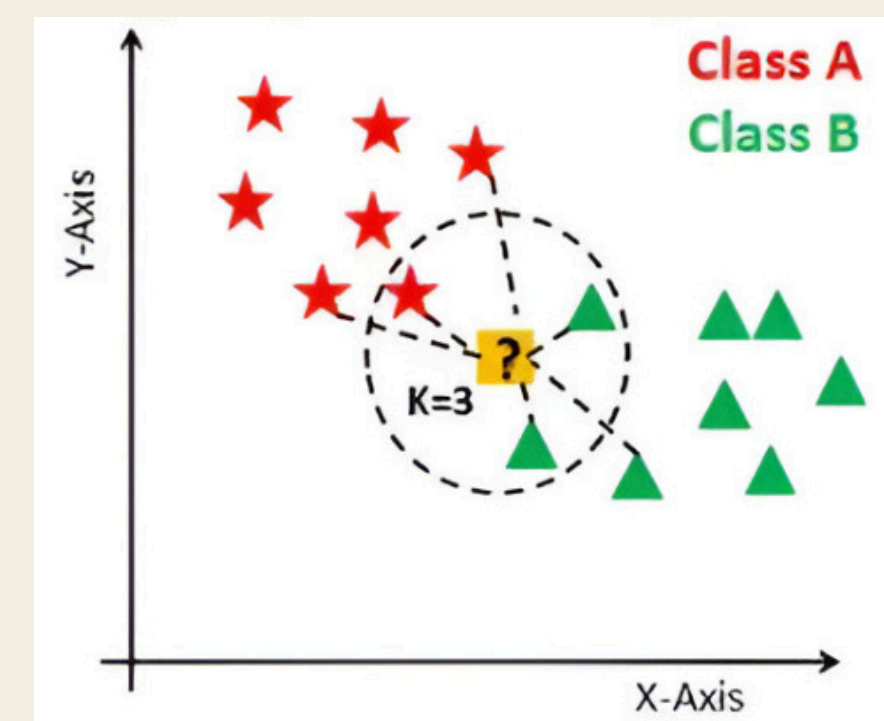
## Step 3

Select k nearest neighbours based on smallest distance.  
(eg. k = 3)



## Step 4

Assign new test data point to the most common class amongst k nearest neighbours



Euclidean  
Distance (2D):

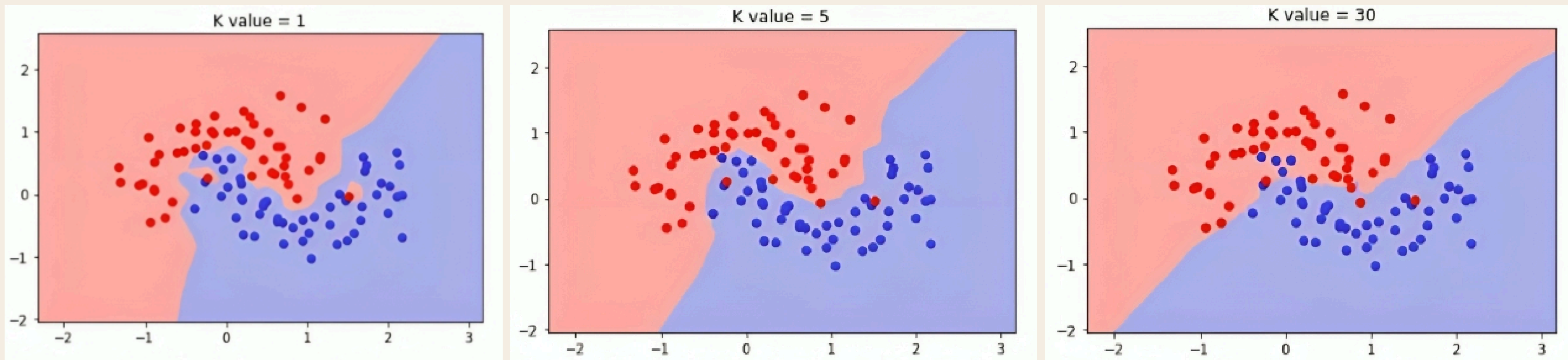
$$\sqrt{(x_1^P - x_1^Q)^2 + (x_2^P - x_2^Q)^2}$$

Euclidean  
Distance (d dim):

$$\sqrt{(x_1^P - x_1^Q)^2 + (x_2^P - x_2^Q)^2 + \dots + (x_d^P - x_d^Q)^2}$$

# How to Choose K?

- No best guidance, so only can trial and error, but the general rule of thumb is  $\sqrt{N}$
- Too small k makes boundary too flexible and sensitive to outliers (overfit)
- Too large k makes boundary too inflexible and tends to underfit
- Always try to keep k as an odd integer to avoid tie between classes



# knn() function in R

```
library(class) # must load this package first!  
  
set.seed(1505) # for reproducibility. can be replaced with any number  
  
knn_pred = knn(train, test, cl, k=1, l = 0, prob = FALSE, use.all = TRUE)
```

so many parameters! don't worry, because not all are required. let's break it down!

# knn() Parameters

```
knn_pred = knn(train, test, cl, k=1, l = 0, prob = FALSE, use.all = TRUE)
```

|       |          |                                                                                                                                                                                                                                        |
|-------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| train | required | <p>A matrix or data frame containing your <u>training data features</u> only (predictor variables).</p> <ul style="list-style-type: none"><li>• <b>Must only contain numerical variables!</b></li></ul>                                |
| test  | required | <p>A matrix or data frame containing the <u>test data features</u> (the data you want to make predictions for).</p> <ul style="list-style-type: none"><li>• <b>Must have the exact same columns as train (in same order)</b></li></ul> |
| cl    | required | <p>A <b>factor</b> vector containing the <b>class labels (cl)</b> for the training data (response variable).</p> <ul style="list-style-type: none"><li>• <b>Length must equal the number of rows in train</b></li></ul>                |

Do I need to memorize this for exam?

**Q:** No! During exam, just enter **?knn** or **help(knn)** after loading library(class), and you can get the exact info here! (works for both RStudio and RGui)

# knn() Parameters

```
knn_pred = knn(train, test, cl, k=1, l = 0, prob = FALSE, use.all = TRUE)
```

|         |                                   |                                                                                                                                                                                                                                                                                                  |
|---------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k       | optional (default k = 1)          | <p>The number of nearest neighbors to consider when making predictions.</p> <ul style="list-style-type: none"><li>• Use <u>odd numbers</u> for binary classification</li></ul>                                                                                                                   |
| l       | optional (default l = 0)          | <p>Minimum winning vote for a knn to output a definite decision.</p> <ul style="list-style-type: none"><li>• Eg. If test point has votes [A, A, B, B, B], but l = 4 (need at least 4 winning votes but there's only 3 B's). Output will be &lt;NA&gt; instead of B.</li></ul>                    |
| prob    | optional (default prob = FALSE)   | <p>Whether to return the proportion of votes for the winning class.</p> <ul style="list-style-type: none"><li>• Eg. If test point has votes [A, A, B, B, B]. Output of knn_pred will show B (winning class) and 0.6 (proportion of winning class)</li></ul>                                      |
| use.all | optional (default use.all = TRUE) | <p>Controls tie-breaking when multiple neighbors are at the same distance.</p> <ul style="list-style-type: none"><li>• TRUE: Use <u>all neighbors</u> at the k-th distance (might use &gt; k neighbors)</li><li>• FALSE: <u>Randomly selects</u> among tied neighbors to use exactly k</li></ul> |

# Splitting Train and Test Data

```
# 1. Set seed to make result reproducible
```

```
set.seed(1505)
```

```
# Eg. Split a total of N data points into: M points for train, K points for test
```

```
# 2. Randomly pick K indices to be the test set
```

```
test = sample(1:N, K)
```

```
train.X = X[-test,] # ALL rows EXCEPT test indices → training features
```

```
test.X = X[test,] # ONLY test indices → test features
```

```
train.Y = Y[-test] # ALL labels EXCEPT test indices → training labels
```

```
test.Y = Y[test] # ONLY test indices → test labels
```

# Standardization

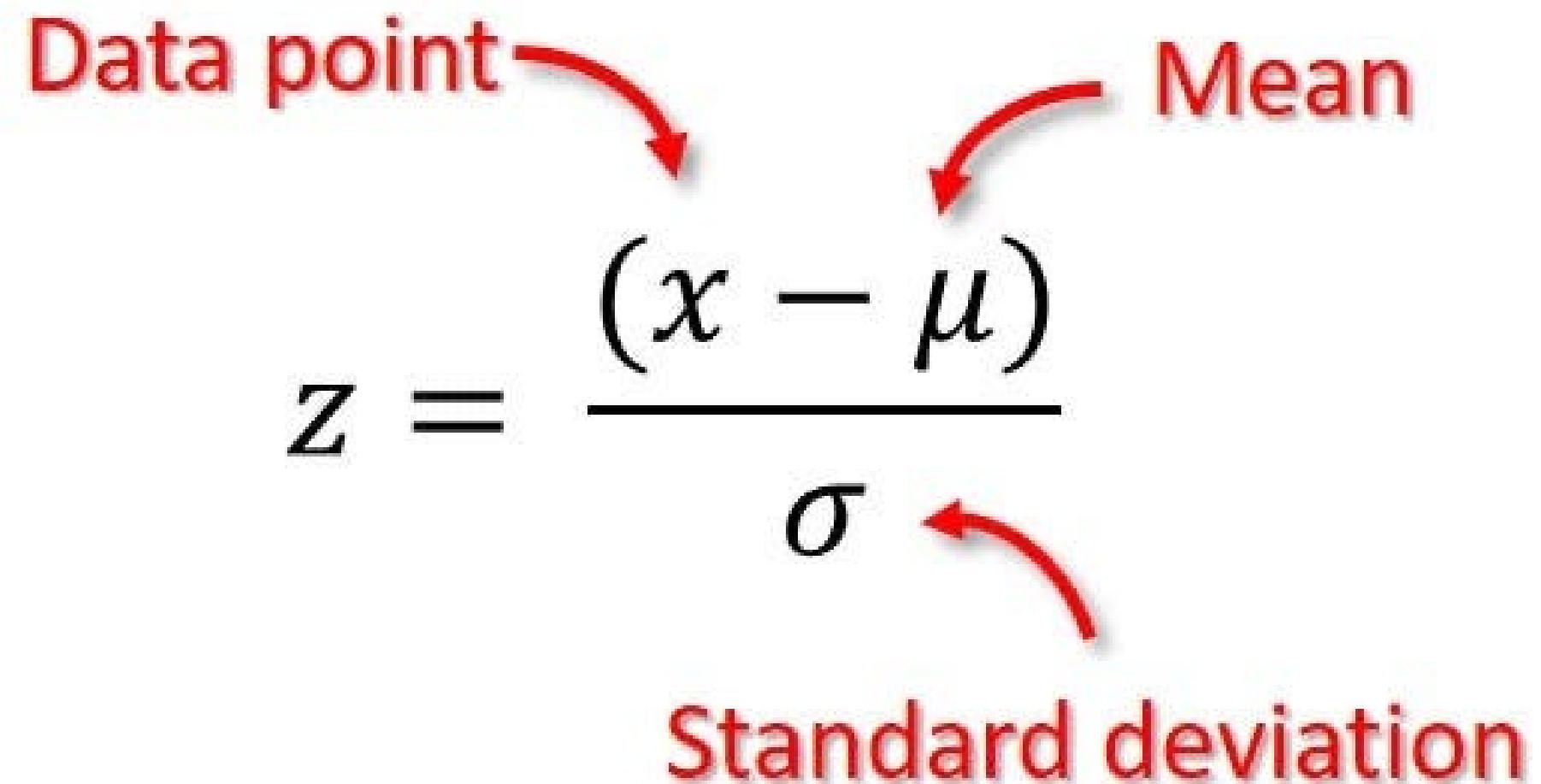
- Ensure all features contribute fairly
- Prevents larger numbers (e.g., salary) from overshadowing smaller ones (e.g., age).

| # | Emp  | Age | Salary |
|---|------|-----|--------|
| 1 | Emp1 | 44  | 73000  |
| 2 | Emp2 | 27  | 47000  |
| 3 | Emp3 | 30  | 53000  |
| 4 | Emp4 | 38  | 62000  |
| 5 | Emp5 | 40  | 57000  |
| 6 | Emp6 | 35  | 53000  |
| 7 | Emp7 | 48  | 78000  |

Try calculating the Euclidean distance. You'll observe that salary dominates the distance metric because of its larger magnitude, essentially drowning out the contribution from age.

# Standardization

Converting data points to Z-scores



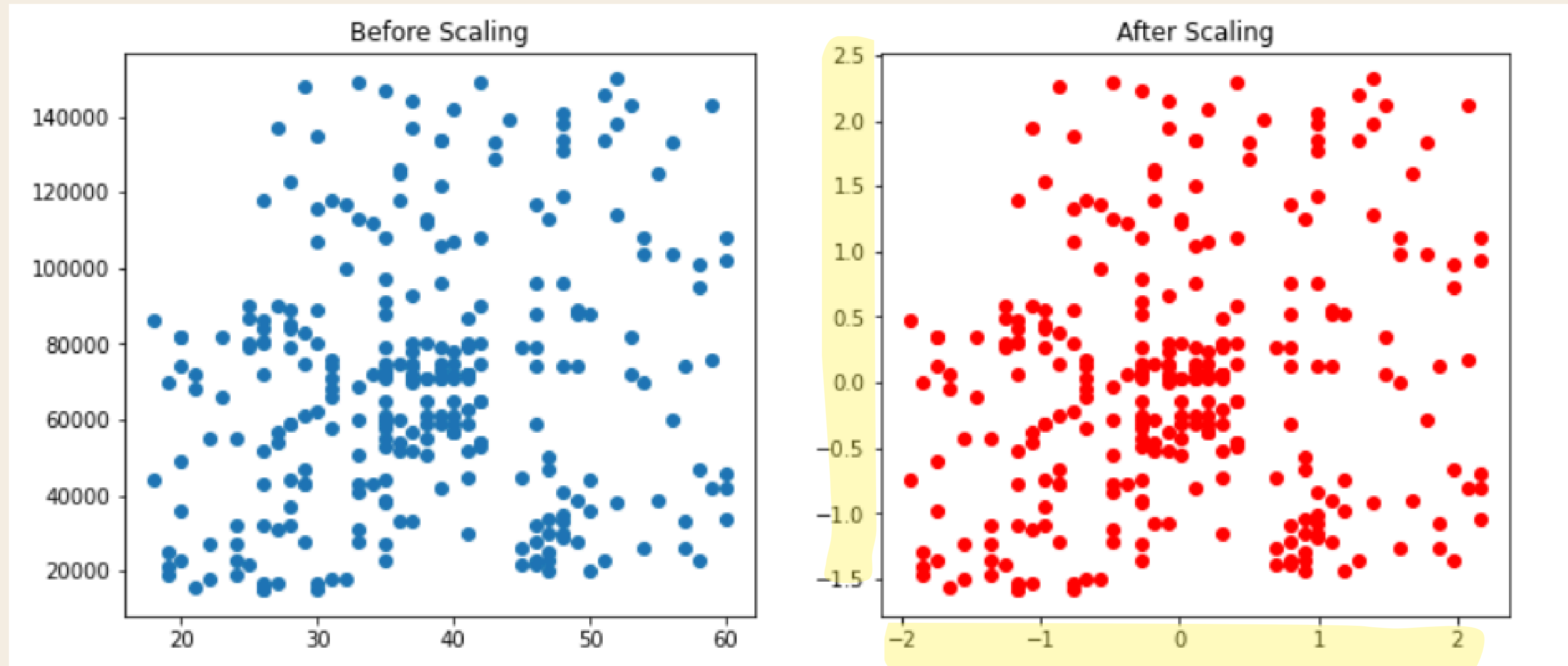
The diagram shows the Z-score formula with three red labels and arrows pointing to the corresponding parts of the equation:

- Data point** points to  $x$  in the numerator.
- Mean** points to  $\mu$  in the numerator.
- Standard deviation** points to  $\sigma$  in the denominator.

$$Z = \frac{(x - \mu)}{\sigma}$$

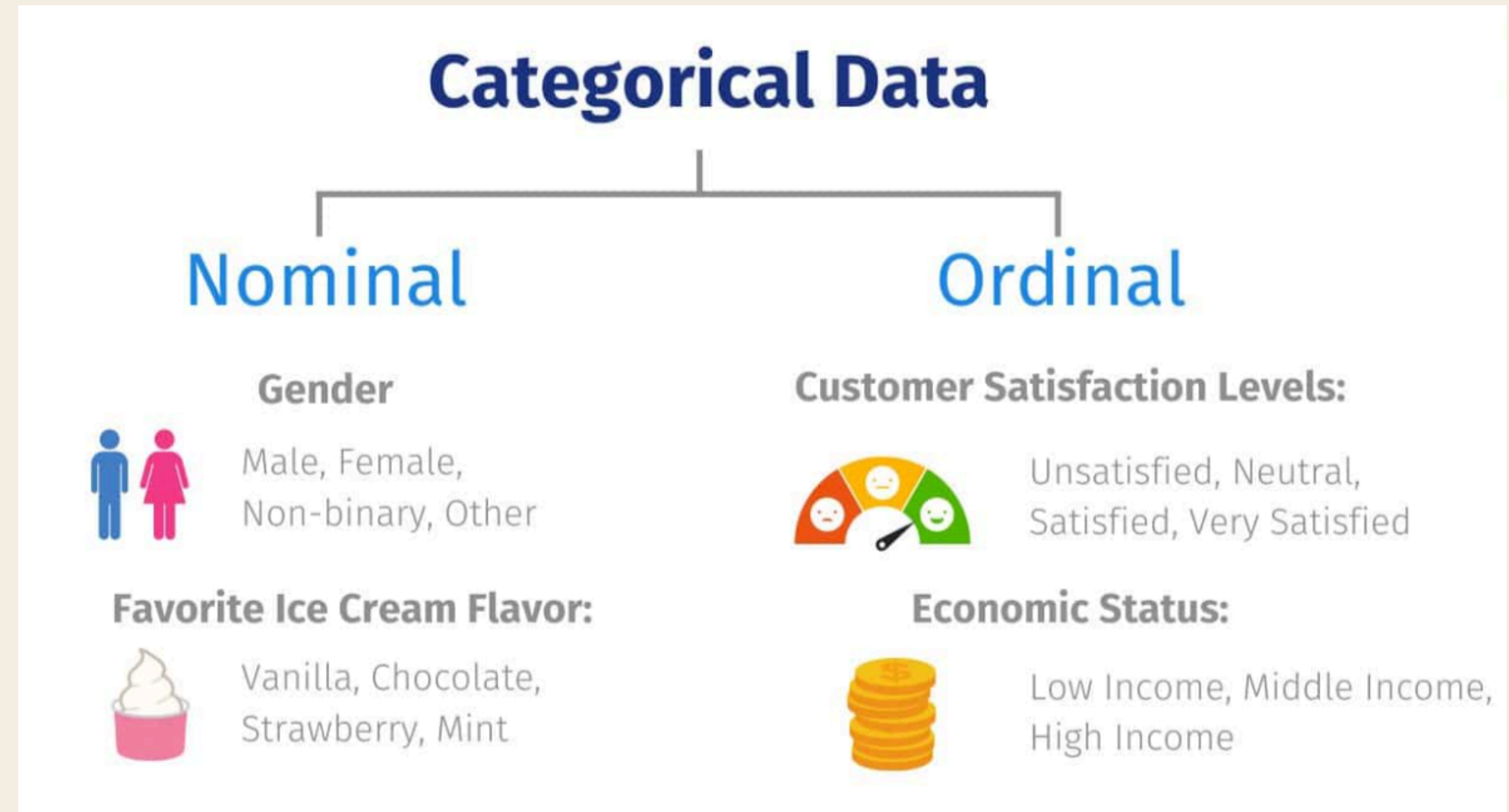
# Standardization

Converting data points to Z-scores



# Which data type to standardize?

- Only numerical variables
  - Includes Quantitative Variables
  - And possibly ordinal (ordered)
- categorical variables
- *Avoid Standardizing Nominal Categorical Variables*



# SPLIT OR STANDARDIZE FIRST?

*Usually:*

*Split then Standardize to avoid Data Leakage!*

*But for DSA1101 (Introductory Course):*

*Follow what the question asks. But usually standardize first then split.*

# STANDARDIZE THEN SPLIT EXAMPLE

```
### Q: Standardize the input features

# remove the first column X since it provides no information
caravan = caravan[,-1]

# scaling all the data set, except the last column
standardized.X = scale(caravan[,-86])

head(standardized.X)
```

```
n = dim(caravan)[1] # sample size = 5822

set.seed(5)

test = sample(1:n, 2000) # sample a random set of 2000 indexes, from 1:n

train.X = standardized.X[-test ,] # training set
test.X = standardized.X[test ,] # test set
train.Y = caravan$Purchase[-test] # response for training set
test.Y = caravan$Purchase[test] # response for test set
```

# !! BUT OUTSIDE THIS COURSE !!

*Split then Standardize to avoid Data Leakage!*

```
# Split first
set.seed(5)
test = sample(1:n, 2000)
train.X = caravan[-test, -c(1, 86)] # Remove first and last column
test.X = caravan[test, -c(1, 86)]
train.Y = caravan$Purchase[-test]
test.Y = caravan$Purchase[test]

# Scale using training set parameters only
train.X.scaled = scale(train.X)

# Apply training set's mean and SD to test set
test.X.scaled = scale(test.X,
                      center = attr(train.X.scaled, "scaled:center"),
                      scale = attr(train.X.scaled, "scaled:scale"))
```

*(1) Split Train Test → (2) Standardize Train →*

*(3) Use Train scale (mean, std) to Standardize Test*

# !! BUT OUTSIDE THIS COURSE !!

*Use mean and SD of train to standardize test*

```
# Scale using training set parameters only
train.X.scaled = scale(train.X)

# Apply training set's mean and SD to test set
test.X.scaled = scale(test.X,
                      center = attr(train.X.scaled, "scaled:center"),
                      scale = attr(train.X.scaled, "scaled:scale"))
```

## IMPORTANT:

Scaling is essential for KNN because it is a distance-based algorithm (uses Euclidean distance!), especially when features have very different ranges (eg. age and income in this example)

# Confusion Matrix

|                        | Actually Positive (1) | Actually Negative (0) |
|------------------------|-----------------------|-----------------------|
| Predicted Positive (1) | True Positives (TPs)  | False Positives (FPs) |
| Predicted Negative (0) | False Negatives (FNs) | True Negatives (TNs)  |

**Sometimes Negative and Positive will switch places. So please read carefully before answering!**

Accuracy =

$$(TP + TN) / (TP + TN + FP + FN)$$

$$TPR (Recall) = TP / (TP + FN)$$

$$FPR = FP / (FP + TN)$$

$$FNR = FN / (TP + FN)$$

$$Precision = TP / (TP + FP)$$

# Evaluation Metrics

## When to use (Optional)

### Accuracy

- Use when classes are balanced and all errors carry equal cost
- Example: Handwritten digit classification (0–9) with equal samples per class

### TPR / Recall

- Use when missing a positive is costly (minimize false negatives)
- Example: Cancer screening → better to over-flag than miss a real case

### FPR

- Use when false alarms are disruptive or expensive (minimize false positives)
- Example: Spam filtering → a high FPR means legitimate emails wrongly land in spam

### FNR

- Use when you want to directly quantify how often positives slip through ( $FNR = 1 - Recall$ )
- Example: Fraud detection → tracks what fraction of real fraud transactions went undetected

### Precision

Use when false positives are costly and you need to trust your positive predictions

Example: Legal document retrieval → every flagged document costs lawyer time, so surfaced results must be relevant

# Extracting Probabilities

**probabilities** returns the winning class's vote share,

**fixed\_probabilities** returns Positive class's vote share.

```
library(class)

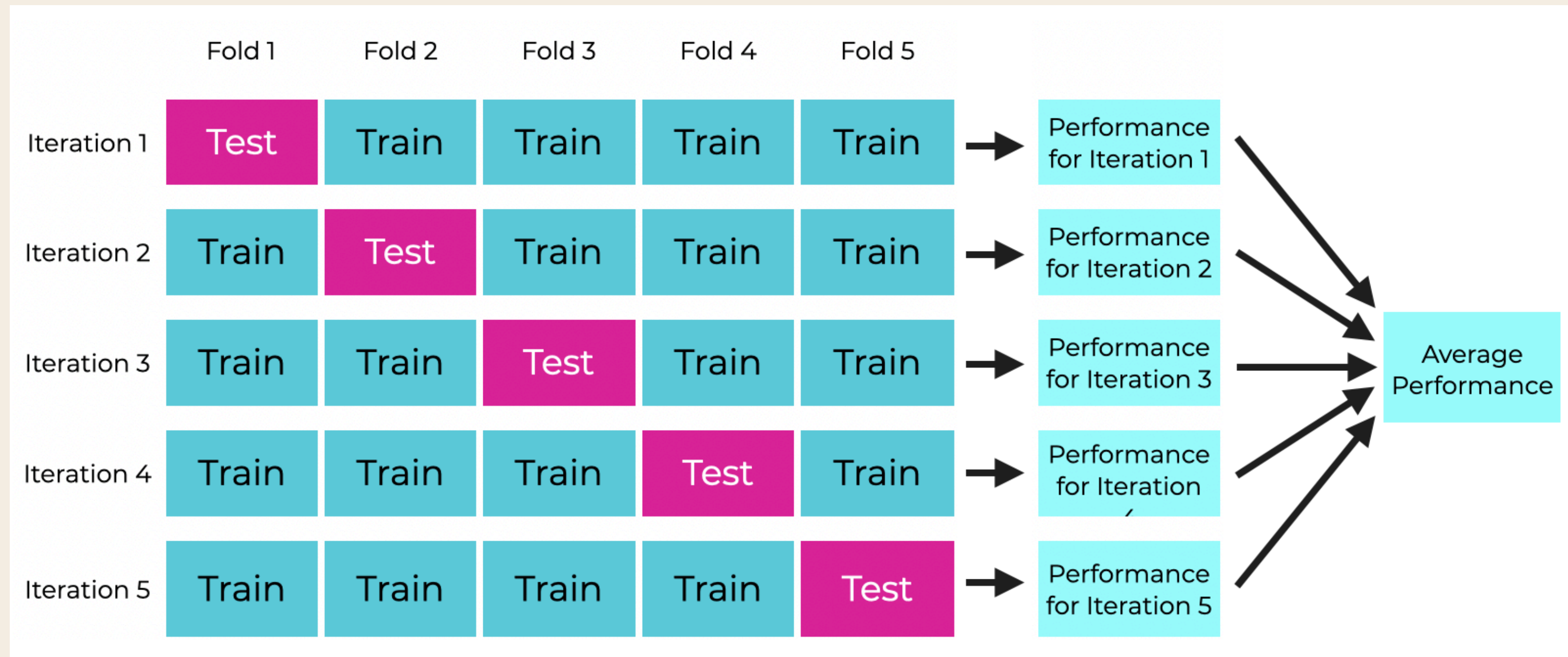
knn.pred <- knn(train.X, test.X, train.Y, k = 3, prob = TRUE)

probabilities <- attr(knn.pred, "prob")

# KNN prob attribute always returns the probability for the winning class,
# not always for "Positive"
fixed_probabilities <- ifelse(
  knn.pred == "Positive",
  probabilities, # TRUE
  1 - probabilities # FALSE
)
```

# N-fold Cross Validation

splits your data into N chunks, trains and evaluates the model N times (each time using a different chunk as the test set), then averages the results for a more reliable performance estimate.



# ON-SITE QUESTIONS

# ON-SITE QUESTION

The table below provides a training data set containing six observations, three predictors, and one qualitative response variable,  $Y$ .

| Obs | $X_1$ | $X_2$ | $X_3$ | $Y$   |
|-----|-------|-------|-------|-------|
| 1   | 0     | 3     | 0     | Red   |
| 2   | 2     | 0     | 0     | Red   |
| 3   | 0     | 1     | 3     | Red   |
| 4   | 0     | 1     | 2     | Green |
| 5   | -1    | 0     | 1     | Green |
| 6   | 1     | 1     | 1     | Red   |

Suppose we wish to use this data set to make a prediction for a new observation with response  $Y$  given that it has  $X_1 = X_2 = X_3 = 0$  using  $K$ -nearest neighbors.

- (a) Compute the Euclidean distance between each observation and the test point,  $X_1 = X_2 = X_3 = 0$ .
- (b) What is our prediction with  $K = 1$ ? Why?
- (c) What is our prediction with  $K = 3$ ? Why?
- (d) If the Bayes decision boundary (the gold standard decision boundary) in this problem is highly non-linear, then would we expect the best value for  $K$  to be large or small? Why?

Discuss with your group! Time limit: 5 mins

# ON-SITE ANSWER (A)

$$\sqrt{(x_1^P - x_1^Q)^2 + (x_2^P - x_2^Q)^2 + \dots + (x_d^P - x_d^Q)^2}$$

| Obs | $X_1$ | $X_2$ | $X_3$ | $Y$   | Euclidean dist. |
|-----|-------|-------|-------|-------|-----------------|
| 1   | 0     | 3     | 0     | Red   | 3.00            |
| 2   | 2     | 0     | 0     | Red   | 2.00            |
| 3   | 0     | 1     | 3     | Red   | 3.16            |
| 4   | 0     | 1     | 2     | Green | 2.24            |
| 5   | -1    | 0     | 1     | Green | 1.41            |
| 6   | 1     | 1     | 1     | Red   | 1.73            |

# ON-SITE ANSWER (B)

| Obs | $X_1$ | $X_2$ | $X_3$ | $Y$   | Euclidean dist. |
|-----|-------|-------|-------|-------|-----------------|
| 1   | 0     | 3     | 0     | Red   | 3.00            |
| 2   | 2     | 0     | 0     | Red   | 2.00            |
| 3   | 0     | 1     | 3     | Red   | 3.16            |
| 4   | 0     | 1     | 2     | Green | 2.24            |
| 5   | -1    | 0     | 1     | Green | 1.41            |
| 6   | 1     | 1     | 1     | Red   | 1.73            |

**ANS: Green**

Closest neighbour to  $X_1 = X_2 = X_3 = 0$  is Observation 5.

We classify majority class (which is Green) to new point.

# ON-SITE ANSWER (C)

| Obs | $X_1$ | $X_2$ | $X_3$ | $Y$   | Euclidean dist. |
|-----|-------|-------|-------|-------|-----------------|
| 1   | 0     | 3     | 0     | Red   | 3.00            |
| 2   | 2     | 0     | 0     | Red   | 2.00            |
| 3   | 0     | 1     | 3     | Red   | 3.16            |
| 4   | 0     | 1     | 2     | Green | 2.24            |
| 5   | -1    | 0     | 1     | Green | 1.41            |
| 6   | 1     | 1     | 1     | Red   | 1.73            |

**ANS: Red**

Closest three neighbour to  $X_1 = X_2 = X_3 = 0$  are Observation 2, 5 and 6.

We classify majority class (which is Red) to new point.

# ON-SITE ANSWER IN R (A,B,C)

```
##### ONSITE QUESTION CODE (OPTIONAL) #####|

### STEP 1: Set up Data Frame
X1 <- c(0, 2, 0, 0, -1, 1)
X2 <- c(3, 0, 1, 1, 0, 1)
X3 <- c(0, 0, 3, 2, 1, 1)
Y <- c("Red", "Red", "Red", "Green", "Green", "Red")

df <- data.frame(X1, X2, X3, Y)
df

### STEP 2: Get Euclidean Distance
df$distance <- sqrt(
  df$X1^2 + df$X2^2 + df$X3^2
)

### STEP 3: Order Data Frame based on Euclidean Distance
df <- df[order(df$distance),]
df

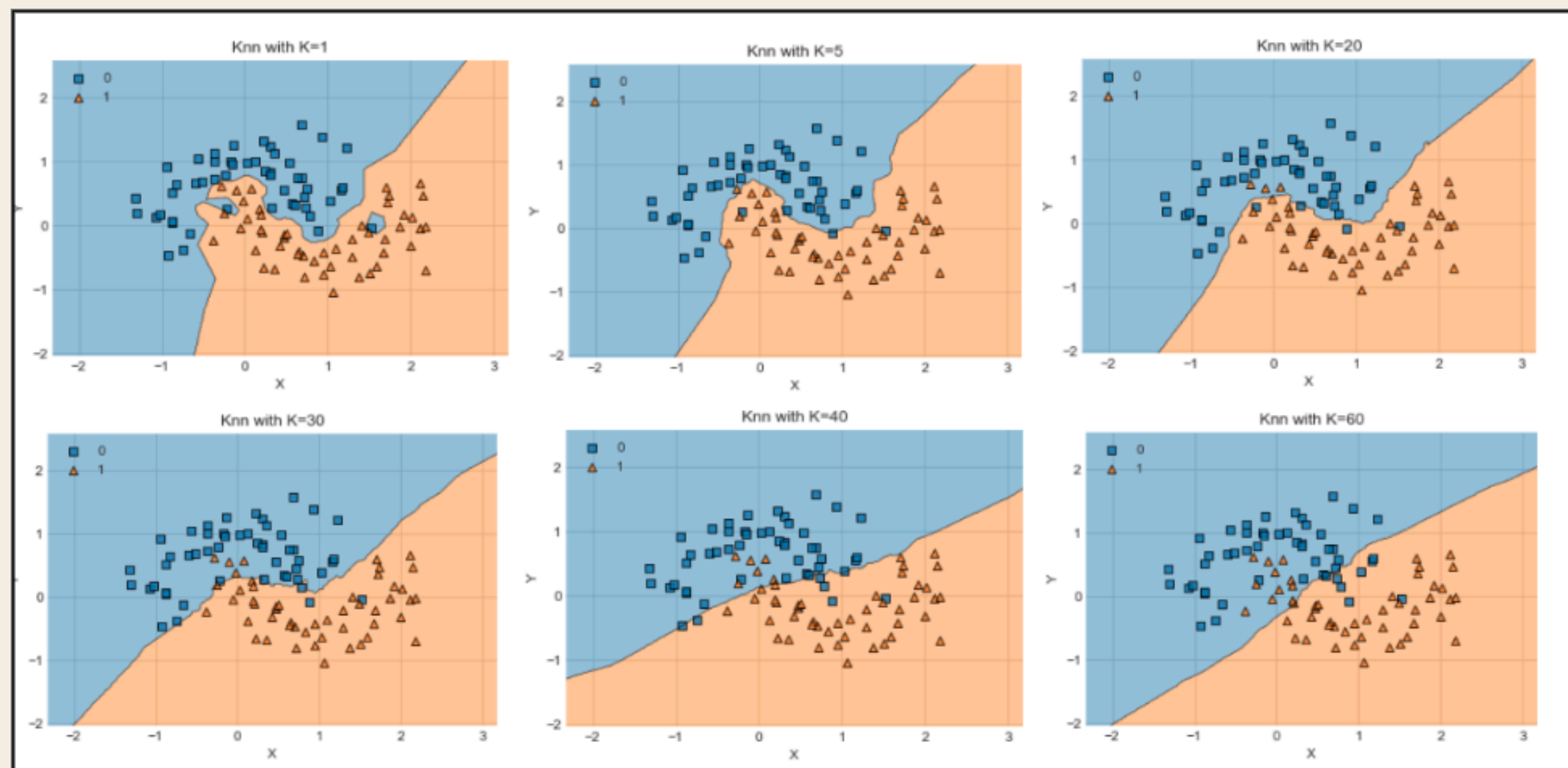
### STEP 4: Class Label for K = 1
df[1,"Y"] # Green

### STEP 5: Class Label for K = 3
max(df[1:3, "Y"]) # Red
```

# ON-SITE ANSWER (D)

Q: If the Bayes decision boundary (the gold standard decision boundary) in this problem is highly non-linear, then would we expect the best value for  $K$  to be large or small? Why?

Ans: A small value for  $K$ , since it translates to a more flexible classification method



# OFF-SITE QUESTIONS

**Let's go to code!**

# MIDTERMS PAST PAPERS



Thank you TA Jin  
for compiling!

[https://drive.google.com/drive/folders/10MTJtBxbYA1  
bRwHmmwGnICUSR5FsZ1ur?usp=sharing](https://drive.google.com/drive/folders/10MTJtBxbYA1bRwHmmwGnICUSR5FsZ1ur?usp=sharing)

# EXTRA KNOWLEDGE: KNN

- KNN Imputer - for missing values in the data
- KNN and Curse of Dimensionality
- KNN Regression
- Feature scaling to make convergence faster

# THANK YOU!



Weekly Slides & Code will be shared on Github (★ star it for bookmark):

<https://github.com/kr7001/DSA1101-teaching-materials>